

Reviewer Pack — Forman-Ricci Overlap-Gap of k-Clique Minterm DNF at $v=10$

Entry c7c90e3a1f6a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 10:15:32 UTC

Forman-Ricci Overlap-Gap of k-Clique Minterm DNF at $v=10$

Entry ID: c7c90e3a1f6a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 10:15:32 UTC

1. Conjecture

Field A (mathematical branch): Forman-Ricci curvature (combinatorial Bochner on minterm hypergraphs) **Field B** (complexity object): Monotone Boolean functions: the k-clique minterm DNF F^*_v on K_v

Statement:

Let F^*_v be the k-clique minterm DNF with $k=\lfloor \log_2 v \rfloor$, viewed as a hypergraph H_v whose vertices are $E(K_v)$ and whose hyperedges are k-cliques represented as their $C(k,2)$ -element edge-sets. Define the max-normalized Forman-Ricci curvature of a minterm T by $\text{ric}(T) := |T| / \max\{|T' \in F_v, T' \neq T\} |T \cap T'|\}$ and let $\mu(F^*_v) := \min_T \text{ric}(T)$. Conjecture: $\mu(F^*_v) \geq 1 + 1/(k-1)$ for every $v \in \{10, 16, 20, 24\}$; in particular $\mu(F^*_{10}) \geq 2$, and for 30 random k-uniform null hypergraphs H_{rand} on $V=E(K_v)$ with the same hyperedge count, $\mu(F^*_v) > \mu(H_{\text{rand}})$ in at least 25 of 30 seeds. A single (v, seed) with the F^*_v bound violated, or with $\mu(F^*_v) \leq \mu(H_{\text{rand}})$ in more than 5 null seeds, refutes the conjecture.

Rationale (proposer's reasoning):

Two distinct k-cliques in K_v share at most $k-1$ vertices, hence at most $C(k-1, 2)$ edges, so by vertex-transitivity $\text{ric}(T)$ equals $C(k, 2)/C(k-1, 2) = k/(k-2)$ for every minterm — a clean Forman invariant that survives 10%-perturbation by deletion since the maximizing overlap pair persists. This anchors SC2 (positivity at $v=10$ with explicit value 2) and gives the Bochner-Weitzenböck base case from which the full $v/4$ bound is meant to inflate via overlap-gap iteration. The random-hypergraph null check separates the clique-DNF signal from baseline overlap statistics, ensuring the curvature reflects the monotone-Boolean structure rather than generic k-uniform geometry.

Taxonomy category: forman_ricci_dnf (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d7bbf9e0c0015586

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each $v \in \{10, 16, 20, 24\}$ with $k = \lceil \log_2 v \rceil$, compute $\mu_{\text{struct}} = C(k, 2) / \text{max_overlap}$ on canonical T_0 and μ_{rand} on 30 seeds of size-matched null k -uniform hypergraphs. Support iff $\mu_{\text{struct}} \geq 1 + 1/(k-1)$ for all 4 v AND $\mu_{\text{struct}} > \mu_{\text{rand}}$ in $\geq 25/30$ seeds per v .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.92	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Forman Ricci curvature hypergraph monotone Boolean - combinatorial Ricci curvature clique complex DNF minterm - Forman curvature k -uniform hypergraph random null model clique

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def comb(n, k):
        if k > n:
            return 0
        result = 1
        for i in range(k):
            result *= (n - i)
            result //= (i + 1)
        return result

    def max_overlap(T, F_v):
        max_overlap = 0
        for T_prime in F_v:
            if T != T_prime:
                overlap = sum(1 for _ in set(T) & set(T_prime))
                max_overlap = max(max_overlap, comb(overlap, 2))
        return max_overlap

    def forman_ricci_curvature(T, F_v):
```

```

max_overlap_val = max_overlap(T, F_v)
if max_overlap_val == 0:
    return float('inf')
return comb(len(T), 2) / max_overlap_val

def generate_null_hypergraph(v, k):
    edges = list(combinations(range(1, v+1), 2))
    null_edges = random.sample(edges, comb(v, 2) - comb(k, 2))
    return [frozenset(e) for e in null_edges]

results = []
for v in {10, 16, 20, 24}:
    k = math.ceil(math.log2(v))
    F_v = list(combinations(range(1, v+1), k))

    T_0 = frozenset(e for e in combinations(range(1, k+1), 2))
    max_overlap_val = max_overlap(T_0, F_v)
    mu_struct = comb(k, 2) / max_overlap_val
    results.append(mu_struct)

    if mu_struct < 1 + 1/(k-1):
        return {
            "metric_name": "mu_struct",
            "metric_value": mu_struct,
            "instances_tested": len(F_v),
            "n_max": k,
            "conjecture_holds": False,
            "counterexample": f"v={v}, k={k}"
        }

    null_hypergraphs = generate_null_hypergraph(v, k)
    mu_rand_count = 0
    for H_rand in null_hypergraphs:
        mu_rand_val = forman_ricci_curvature(H_rand[0], F_v)
        if mu_rand_val < mu_struct:
            mu_rand_count += 1

    if mu_rand_count > 5:
        return {
            "metric_name": "mu_struct",
            "metric_value": mu_struct,
            "instances_tested": len(F_v),
            "n_max": k,
            "conjecture_holds": False,
            "counterexample": f"v={v}, k={k}"
        }

return {
    "metric_name": "mu_struct",
    "metric_value": sum(results) / len(results),
    "instances_tested": 4 * comb(v, 2) - comb(k, 2),
    "n_max": max([math.ceil(math.log2(v)) for v in {10, 16, 20, 24}]),
    "conjecture_holds": True,
    "counterexample": ""
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean)**2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r >= 1 + 1/(math.ceil(math.log2(v)) - 1) for v in {10, 16, 20, 24}):

    if all(r >= 1 + 1/(math.ceil(math.log2(v)) - 1) for r in results for v in {10, 16, 20, 24}):
        print(f"RESULT: SUPPORTED mean={mean:.2f} std={std:.2f} support_fraction={support_fraction}")
    elif sum(r >= 1 + 1/(math.ceil(math.log2(v)) - 1) for r in results for v in {10, 16, 20, 24}):
        print(f"RESULT: SUPPORTED mean={mean:.2f} std={std:.2f} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results, start=1) if not all(
            print(f"RESULT: FALSIFIED counterexample=\"first_failing_seed\" first_failing_seed={first_failing_seed}"))

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_58bf8af9.py", line 100, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_58bf8af9.py", line 55, in run_trial
    mu_struct = comb(k, 2) / max_overlap_val
                  ~~~~~^~~~~~
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ZeroDivisionError before producing any data, so neither the structural bound nor the null comparison was evaluated. Per the rules, a crash yields INCONCLUSIVE. | next: Guard against `max_overlap_val=0` (e.g., set `μ_struct=∞` or skip when no other minterm overlaps) and re-run the trials for `v∈{10,16,20,24}`.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	265689
2	preregistration	claude_max	opus	0	0	5283
3	novelty	claude_max	opus	0	0	3528
4	novelty	claude_max	opus	0	0	5506
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17395
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18592
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18487
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19039
9	judge	claude_max	opus	0	0	3838

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 357359 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/c7c90e3a1f6a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c7c90e3a1f6a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c7c90e3a1f6a.tar.gz` (if generated)